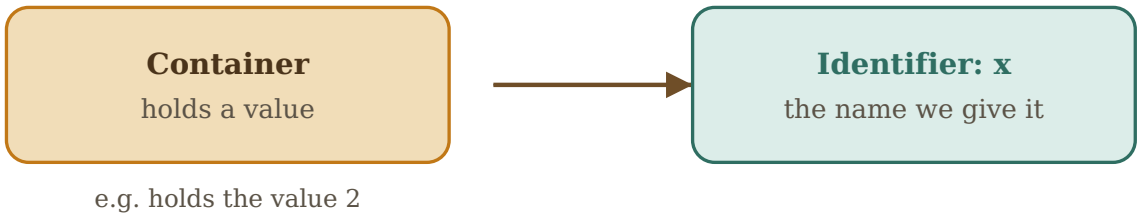
CORE JAVA SERIES — DAY 2

Variables, Data Types & Binary Representation

From what a variable actually is, to the four primitive data type families, to how Java stores negative numbers and decimals as raw bits in memory.

What Is a Variable?

Any program is written to **do** something — print to screen, perform a calculation, build an application. The moment your program needs to hold onto a value (not just print it once), you need a place in memory to store it. That memory location is what we call a **variable**.



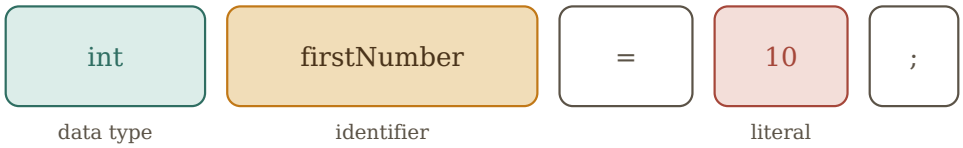
A variable is a container in memory that holds a value; the name we give that container is its identifier.

Example: building a calculator that adds two numbers. You'd need two containers — `firstNumber` and `secondNumber` — to hold the inputs before adding them. The names should be as **descriptive** as possible so the code explains itself.

> **Identifier:** The name used to refer to a container's value. In `firstNumber = 10`, `firstNumber` is the identifier.

Declaring a Variable — Syntax

To declare a variable in Java, you write, in order: **data type** → **identifier** → `=` → **value** → `;`



Every Java statement ends with a semicolon — this marks the end of the line, just like in C++.

> **Java is Statically Typed:** You must declare the data type of a variable before/at compile time — Java always knows in advance what kind of value a container will hold.

Data Types: The Big Picture

Java data types split into two broad families: **Primitive** and **Non-Primitive**. This series covers primitives first — non-primitives come later, once we study OOP.

Primitive data types fall into four categories:

Integer	Floating Point	Character	Boolean
byte, short, int, long	float, double	char	boolean

Integer Types: byte, short, int, long

All four hold whole numbers — they differ only in **range**, i.e. how large a number they can represent. Range increases in this order: `byte` < `short` < `int` < `long`.

Type	Width	Range
byte	8 bits	−128 to +127
short	16 bits	−32,768 to +32,767
int	32 bits	≈ −2.1 billion to +2.1 billion
long	64 bits	in the quadrillions

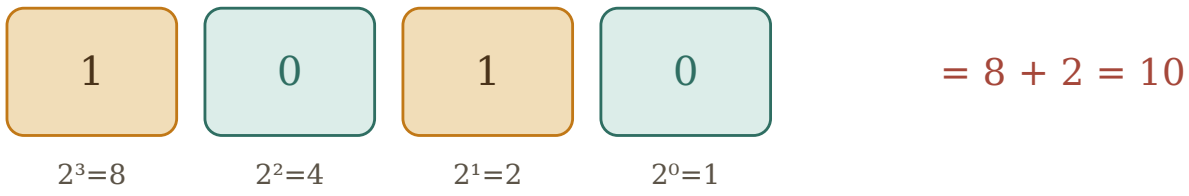
Why Bit-Width Determines Range

With **n bits**, you can form 2^n unique combinations of 0s and 1s. The largest number representable is therefore $2^n - 1$ (since the smallest is 0).

> **Example:** With 8 bits, total combinations = $2^8 = 256$. Largest unsigned value = 255. But since **every number in Java is signed** (can be positive or negative), that range of 256 values is split across negative and positive — giving byte its actual range of −128 to +127, not 0 to 255.

Converting Decimal to Binary (Quick Refresher)

Decimal (base 10) uses digits 0–9; Binary (base 2) uses only 0 and 1. Each position in a binary number represents a power of 2: 2^0 (ones), 2^1 (twos), 2^2 (fours), 2^3 (eights), and so on, increasing left to right.



1010 in binary = 10 in decimal (add up the positions where a 1 appears).

Floating Point Types: float & double

For decimal (non-whole) numbers, Java uses **float** (32 bits, single precision) and **double** (64 bits, double precision). Double has a much larger range and far greater precision.

Type	Width	Precision
float	32 bits	Single precision
double	64 bits	Double precision

> **Why double is preferred:** Modern processors are optimized for double, not float. float is rarely used in real production code — mainly reserved for cases where precision genuinely doesn't matter (e.g. cents in a dollar amount). Every built-in Java math method (sin, cos, etc.) returns a `double`.

> **Syntax note:** A decimal literal like `10.54` is treated as a `double` by default. To store it in a `float` variable, you must suffix it with `f` — e.g. `float f = 10.54f;` — otherwise you get a type-mismatch compile error.

Character Type: char

`char` represents a single **Unicode** character, written in single quotes: `char alphabet = 'a';`. The single quotes tell

the compiler this is a character literal, not an identifier.

How a Character Becomes Binary

A computer can't store `'a'` directly — it must first be converted to an integer, then that integer is converted to binary. Before Java, languages used the **ASCII** standard (0–127, 8 bits) to map English letters and symbols to numbers. Java extended this with **Unicode** — giving every character in every world language (not just English) an integer value — which needs **16 bits** instead of 8.

> **Key fact:** A Java `char` takes **16 bits**, because Java wanted to represent all of the world's languages via Unicode, not just the English alphabet covered by ASCII.

Boolean Type: boolean

The simplest data type — only two possible values: `true` or `false`. Syntax: `boolean b = false;`

> **Unlike C/C++:** In C/C++, any non-zero number is treated as true and 0 as false. In Java, `boolean` only ever means `true` or `false` — you cannot write `b = 0` or `b = 1`.

Literals in Other Number Systems

Besides standard decimal, Java lets you write integer literals in three other number systems, using a prefix:

System	Base	Prefix	Example (for 5)
Binary	2	`0b`	`0b101`
Octal	8	`0`	`05`
Hexadecimal	16	`0x`	`0x5`

Hexadecimal digits run 0–9 then **a–f** for 10–15 (e.g. `0xf` = 15, `0xa` = 10). Octal digits are restricted to 0–7 (writing `08` is a compile error, since 8 is out of range for base 8).

Underscores for Readability

Large numeric literals can use underscores as visual separators, e.g. `long l = 123_456_789;` — the compiler ignores them completely. Restriction: you can't place an underscore directly before/after a decimal point, or right before/after the `e` in scientific notation.

Scientific Notation

Large/small double values can be written in scientific form: `6.022e23` means 6.022×10^{23} (Avogadro's number).

Declaration vs. Definition

So far every example both declared and defined a variable in one line. These are actually two separate steps:

- **Declaration** — `int x;` tells the compiler "reserve a container named x, of type int" (32 bits set aside), without giving it a value yet.
- **Definition** — `x = 4;` later assigns an actual literal value into that reserved container.

This split matters when a value isn't known upfront — e.g. it comes from user input rather than being hard-coded.

Keywords

Keywords are reserved words that a programming language sets aside for its own syntax (e.g. `class`, `public`, `static`, `void`). You cannot use a keyword as the name of your own variable or, later, a function — the compiler would get confused about what you mean.

> **Fun fact:** Java has **68 keywords** in total. A few — like `goto` and `const` — are reserved but never actually used in the language, carried over from C/C++ conventions during Java's early design.

Comments — text the compiler ignores, meant only for humans — come in two forms: single-line (`// comment`) and multi-line (`/* comment */`).

Active Recall — Part 1

Q1. What is a variable?

A. A named container in computer memory that holds a value; the name is called an identifier.

Q2. Why does Java require you to specify a data type when declaring a variable?

A. Because Java is statically typed — the compiler must know in advance what kind of value (and how many bits) a variable will hold, before/at compile time.

Q3. With n bits, how many unique values can be represented, and what's the largest?

A. 2^n unique combinations; the largest representable (unsigned) value is $2^n - 1$.

Q4. Why is byte's range −128 to +127 rather than 0 to 255?

A. Because every number in Java is signed — the 256 total combinations of 8 bits are split between negative and positive values.

Q5. Why is double generally preferred over float in real projects?

A. Modern hardware is optimized for double's precision and calculations; float is single precision and rarely used except where precision genuinely doesn't matter.

Q6. Why does a Java char take 16 bits instead of the 8 bits ASCII used?

A. Because Java uses Unicode to represent characters from every world language, not just the English alphabet that ASCII covers.

Q7. What's the difference between declaring and defining a variable?

A. Declaration (`int x;`) reserves a typed container without a value; definition (`x = 4;`) assigns an actual value into it.

Storing Negative Numbers: Two's Complement

Positive numbers convert to binary directly. But how does a computer — which only understands 0s and 1s — represent a negative number like −42? Java (like most systems) uses a technique called **Two's Complement**.

The Two's Complement Process

- **Step 1:** Ignore the sign and find the binary of the absolute value (e.g. binary of 42 = `00101010`)
- **Step 2 — One's Complement:** Flip every bit (0→1, 1→0) → `11010101`
- **Step 3 — Two's Complement:** Add 1 to the One's Complement result → `11010110`

This final value, `11010110` , is what actually gets stored in memory for −42.



Two's complement in three steps: find the binary, flip every bit, then add 1.

How the Computer Knows It's Negative: MSB

The **Most Significant Bit** (MSB — the left-most bit) acts as the sign indicator: **MSB = 1** means negative, **MSB = 0** means positive. This works cleanly because the largest positive byte value (+127) never needs that left-

most bit set — so a 1 there can only mean "negative."

> **Reading a negative number back:** To convert a stored two's-complement value back to decimal — take its one's complement (flip all bits), add 1, convert to decimal, and prefix a negative sign.

Why Two's Complement (Not Just One's Complement)?

One's complement alone has a flaw: it creates two different representations for zero (`00000000` and `11111111` for "negative zero"), which is mathematically invalid. Two's complement fixes this — when you compute −0 via two's complement, the extra carry bit overflows past the 8-bit width and gets discarded, leaving plain `00000000`. This guarantees zero only ever has one representation.

Storing Floating Point Numbers: IEEE 754

Floating point storage is more involved. A `float` uses **32 bits**, split into three parts per the **IEEE 754** standard:



float = 32 bits total: 1 (sign) + 8 (exponent) + 23 (mantissa).

Encoding Steps (e.g. for 8.125)

- **Step 1:** Convert the number to binary (integer part + fractional part) → `1000.001`
- **Step 2:** Normalize to the form `1.x × 2^exponent` → `1.000001 × 2³`
- **Step 3:** Add the **bias** to the exponent (bias = 127 for float) → 3 + 127 = 130, converted to binary
- **Step 4:** Place sign bit, biased exponent, and mantissa (the `x` part after "1.") into the 32-bit layout

> **Why a bias?** The IEEE 754 standard avoids storing negative exponents directly. Adding a bias (127 for float, chosen as $2^{(n-1)} - 1$ for an n-bit exponent field) shifts every possible exponent into a positive range, which simplifies hardware comparison and calculation. For `double` (11-bit exponent), the bias is **1023**.

Decoding Formula

To retrieve the original decimal value from stored bits:

```
value = (−1)^sign × (1 + mantissa) × 2^(exponent − bias)
```

The Precision Problem

Some decimal fractions — like **0.7** — never terminate in binary; the bit pattern repeats infinitely (`0.10110011...`). Since only 23 (or 52, for double) mantissa bits are available, the value gets **truncated**. This is why printing `0.7f` with many decimal places shows something like `0.69999998...` instead of an exact 0.7 — the stored value is only the closest possible approximation, not the exact number.

> **Takeaway:** Neither `float` nor `double` can represent every decimal value exactly — this is a fundamental limitation of binary floating-point storage, not a bug. For cases needing exact decimal precision (e.g. financial calculations), Java provides `BigDecimal`, covered later in this series.

double vs float — Same Idea, Bigger Layout

Type	Total Bits	Sign	Exponent	Mantissa	Bias

float	32	1	8 bits	23 bits	127
double	64	1	11 bits	52 bits	1023

Active Recall — Part 2

Q1. What are the three steps of Two's Complement for representing a negative number?

A. Find the binary of the absolute value → flip all bits (One's Complement) → add 1 (Two's Complement).

Q2. How does a computer know whether a stored binary number is negative or positive?

A. By checking the Most Significant Bit (MSB): 1 means negative, 0 means positive.

Q3. Why is Two's Complement used instead of just One's Complement?

A. One's Complement creates two representations of zero (positive and "negative" zero), which is invalid. Two's Complement's extra +1 step causes the carry to overflow and discard, leaving only one valid representation of zero.

Q4. What are the three parts of a 32-bit float's IEEE 754 layout?

A. 1 sign bit, 8 exponent bits, and 23 mantissa bits.

Q5. Why does IEEE 754 add a bias to the exponent?

A. To avoid storing negative exponents directly — the bias (127 for float, 1023 for double) shifts all exponents into a positive range for simpler hardware processing.

Q6. Why does printing 0.7f sometimes show 0.69999998... instead of exactly 0.7?

A. Because 0.7 doesn't terminate in binary — its bit pattern repeats infinitely, so it gets truncated to fit the available mantissa bits, storing only the closest approximation.

Q7. What Java type should you use when you need an exact decimal value (e.g. for financial calculations)?

A. BigDecimal, which avoids the approximation issues inherent to float and double.